Pluggable Backends in LLVM

PreCodeGenCallback and friends

Stefan Gränitz · Berlin LLVM · 2026

Agenda

1. Introduction to pass plugins

What is an LLVM backend? • Plugin interfaces in LLVM • 🎬 Pass plugins

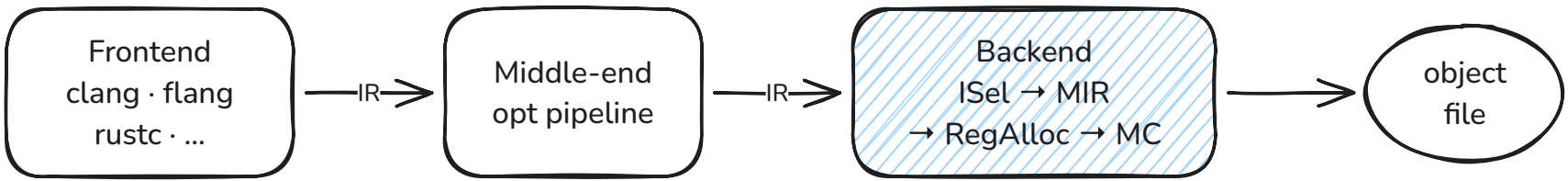
2. PreCodeGenCallback for backends

The new `PreCodeGenCallback` • Why in the pass-plugin API? • 🎬 The hook in llc • 🎬 TPDE in clang

3. Status and outlook

Who adopted it? • What's missing?

What is an LLVM backend?



Codegen, plus:

- calling conventions
- debug info, EH / unwind tables
- inline asm, object files, relocations
- stack maps, CFI, pointer auth

What it cannot do:

- add a new arch: IR depends on it, and `Triple::ArchType` is closed
- offloading: frontends split host and device code

Why would we want a custom backend?

- **compile time:** TPDE (10–20× faster than `-O0`)
- **other output:** llvm-cbe (C backend), llvm-spirv (Khronos SPIRV-LLVM-Translator)
- **capture the final IR:** wllvm / gllvm, `-fembed-bitcode`
- **validation:** arm-tv (Regehr, Lopes et al.) finds backend miscompilations

Plugin interfaces in LLVM

Interface	Loaded by
Clang frontend plugins	`clang -fplugin=`
Legacy-PM plugi`ns	`opt -load` and static ctors like `RegisterTargetPassConfigCallback`
New-PM pass plugins	`clang -fpass-plugin` etc. with `llvmGetPassPluginInfo()`
Other unrelated ones	clang-tidy, clangd, lldb, MLIR dialect plugins, CAS plugin C API

The pass-plugin interface

See my EuroLLVM 2025 talk from last year

```
struct PassPluginLibraryInfo {
  uint32_t APIVersion;         // LLVM_PLUGIN_API_VERSION: 3 on main
  const char *PluginName;
  const char *PluginVersion;
  void (*RegisterPassBuilderCallbacks)(PassBuilder &);
  bool (*PreCodeGenCallback)(Module &, TargetMachine &,
                             CodeGenFileType, raw_pwrite_stream &);  // LLVM 22
  Error (*ParseArguments)(ArrayRef<const char *> Args);              // LLVM 24
};

extern "C" PassPluginLibraryInfo llvmGetPassPluginInfo();  // looked up via dlsym
```

Only works with the New PassManager!

- default in opt pipeline for a while
- still in-progress for the codegen pipeline

the C++ ABI between plugin and host is unchecked → exact LLVM version

Demo 1 · Bye, the minimal pass plugin

`llvm/examples/Bye` prints Bye for all functions 👋

```
# opt / llc (LLVM 24)
opt -load-pass-plugin=Bye.so \
    -passes=goodbye \
    -plugin-arg=Bye,-wave-goodbye \
    hello.ll
```

```
# clang
clang -fpass-plugin=Bye.so \
      -mllvm -wave-goodbye \
      -c hello.c
```

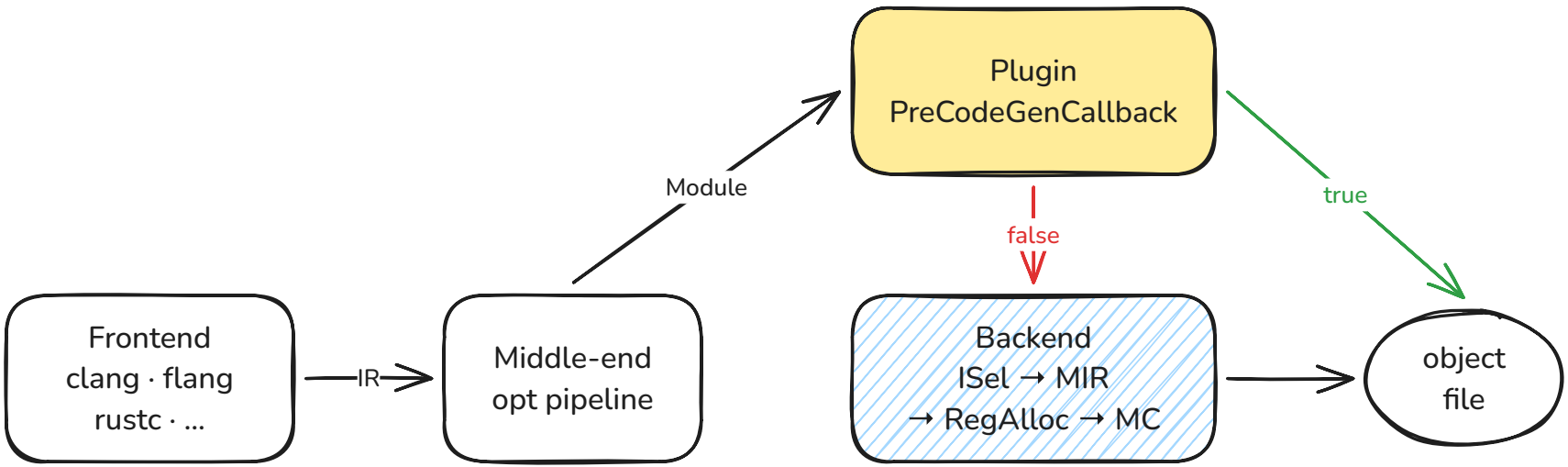
one plugin, two spellings for its option

```
$ head -15 demo/hello.ll
; ModuleID = 'hello.c'
source_filename = "hello.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i8:8:32-i16:16:32-i64:64-i128:128-n32:64-S128
-Fn32"
target triple = "aarch64-unknown-linux-gnu"

; Function Attrs: mustprogress nofree norecurse nosync nounwind willreturn memory(none) uwtable
define dso_local i32 @add(i32 noundef %0, i32 noundef %1) local_unnamed_addr #0 {
  %3 = add nsw i32 %1, %0
  ret i32 %3
}

; Function Attrs: mustprogress nofree norecurse nosync nounwind willreturn memory(none) uwtable
define dso_local noundef range(i32 -2147483648, 2147483645) i32 @main() local_unnamed_addr #0 {
  ret i32 0
}
$ ./demo/demo1.sh
+ clang -O1 -S -emit-llvm -fpass-plugin=./Bye.so -mllvm -wave-goodbye hello.c -o hello.ll
Bye: add
Bye: main
+ opt -load-pass-plugin=./Bye.so -passes=goodbye -plugin-arg=Bye,-wave-goodbye -disable-output hello.ll
Bye: add
Bye: main
$
```

PreCodeGenCallback landed in `#170846`



`true` → skip regular backend and all later plugins

`false` → fall back to LLVM

Why in the pass-plugin API?

It doesn't fit

- pass plugins *add passes* to a pipeline
- this hook *replaces* the pipeline
- no pass, no `PassBuilder`, first one wins

Alternatives (#165257)

- keep it in Clang (original RFC): every frontend needs its own copy
- Me: `addPassesToEmitFile()` would be the right layer, but wrong abstraction
- Eli Friedman: extend `PassPluginLibraryInfo` ✅

pragmatic: loading and distribution are already solved

The missing layer now exists

`llvm::runCodeGenPipeline` · Aiden Grossman, #220305, Sep 2026

```
Error runCodeGenPipeline(TargetMachine &TM, Module &M,
                         raw_pwrite_stream &OS, raw_pwrite_stream *DwoOS,
                         CodeGenFileType CGFT,  ... );
```

- same parameters as the hook, sees the `Module`, legacy and new PM
- clang and flang use it already. Next: llc and LTO?

right layer + right loading mechanism, finally in one place?

Demo 2 · the hook in llc

Bye has a `PreCodeGenCallback` too, behind `-last-words`

```
# normal asm
llc -load-pass-plugin=Bye.so hello.ll

# CodeGen Bye
llc -load-pass-plugin=Bye.so -plugin-arg=Bye -last-words hello.ll

# error
llc ... -filetype=obj hello.ll
```

```
$ ./demo/demo2.sh
+ llc -load-pass-plugin=./Bye.so hello.ll -o -
+ head -n 8
        .file   "hello.c"
        .text
        .globl  add                             // -- Begin function add
        .p2align        2
        .type   add,@function
add:                                    // @add
        .cfi_startproc
// %bb.0:
+ llc -load-pass-plugin=./Bye.so -plugin-arg=Bye,-last-words hello.ll -o -
CodeGen Bye
+ llc -load-pass-plugin=./Bye.so -plugin-arg=Bye,-last-words -filetype=obj hello.ll -o /dev/null
error: last words unsupported for binary output
+ true
+ clang -S -O0 -fpass-plugin=./Bye.so -mllvm -last-words hello.c -o -
CodeGen Bye
+ clang -c -O0 -fpass-plugin=./Bye.so -mllvm -last-words hello.c -o -
error: last words unsupported for binary output
1 error generated.
$
```

Demo 3 · TPDE in clang

The whole plugin, about 40 lines

```
extern "C" PassPluginLibraryInfo llvmGetPassPluginInfo() {
  return {.APIVersion = LLVM_PLUGIN_API_VERSION, .PluginName = "TPDE-LLVM", .PluginVersion = "1",
          .PreCodeGenCallback = [](Module &mod, TargetMachine &,
                                   CodeGenFileType file_type, raw_pwrite_stream &os) {
    if (file_type != CodeGenFileType::ObjectFile) {
      mod.getContext().emitError("TPDE only support compiling to object files");
      return false;
    }
    auto compiler = tpde_llvm::LLVMCompiler::create(mod.getTargetTriple());
    std::vector<uint8_t> buf;
    if (compiler && compiler->compile_to_elf(mod, buf)) {
      os.write(reinterpret_cast<char *>(buf.data()), buf.size());
      return true;
    }
    mod.getContext().diagnose(DiagnosticInfoGeneric{"TPDE compilation failed", DS_Warning});
    return false;
  }};
}
```

Demo 3 · TPDE in clang

TPDE ships `tpde-plugin.so` for LLVM 22+, no patch needed

```
clang -O0 -c -fpass-plugin=tpde-plugin.so \
      -Werror=backend-plugin hello.c -o hello.o
clang -fuse-ld=lld hello.o -o hello && ./hello

clang -O0 -S -fpass-plugin=tpde-plugin.so hello.c   # error: no asm
```

ELF only: x86-64 and AArch64

```
Benchmark 1: clang -O0 -c hello.c -o /dev/null
  Time (mean ± σ):      65.4 ms ±   7.7 ms    [User: 5.3 ms, System: 30.3 ms]
  Range (min … max):    58.2 ms … 105.8 ms    50 runs

  Warning: Statistical outliers were detected. Consider re-running this benchmark on a quiet system witho
ut any interferences from other programs. It might help to use the '--warmup' or '--prepare' options.

Benchmark 2: clang -O0 -c -fpass-plugin=./tpde-plugin.so hello.c -o /dev/null
  Time (mean ± σ):      67.9 ms ±   8.2 ms    [User: 4.3 ms, System: 31.4 ms]
  Range (min … max):    59.5 ms … 103.4 ms    46 runs

Summary
  clang -O0 -c hello.c -o /dev/null ran
    1.04 ± 0.18 times faster than clang -O0 -c -fpass-plugin=./tpde-plugin.so hello.c -o /dev/null
$
$ ./demo/llc-time-trace.sh
+ llc -O0 -filetype=obj -time-trace -time-trace-file=../build/bench/trace-llvm.json ../build/bench/Verifi
er.bc -o /dev/null
+ llc -O0 -filetype=obj -load-pass-plugin=./tpde-plugin.so -time-trace -time-trace-file=../build/bench/tr
ace-tpde.json ../build/bench/Verifier.bc -o /dev/null

                                    LLVM      TPDE
total (wall)                       782ms     140ms
backend (OptModule / TPDE)         599ms      22ms
rest: startup, read + verify IR    183ms     118ms

backend speedup:     26.8x
end-to-end speedup:   5.6x
$
```

Who adopted the PreCodeGenCallback?

Client	Status
llc, clang, flang	✅ LLVM 22+
Rust	🟡 open PR rust#164108: `-Zllvm-tpde=try|only`
LTO, lld, distributed ThinLTO	❌ open PR #231516
clang-repl	❌ JIT path, TPDE in ORC instead
Julia, Zig, Swift	no evidence yet

What's missing?

- **LTO**: open PR #231516 (regular, in-process and distributed ThinLTO)
- plugins don't get the hook, that are **linked statically** in LLVM (`LLVM_${name}_LINK_INTO_TOOLS`)
- **plugin arguments** are inconsistent in clang: no `-fpass-plugin-arg` , only `-mllvm`
- **file output only** as the API has a `raw_pwrite_stream` (no implicit JIT support)

Related

New Pass-Manager in Codegen:

- Plugin-support so far not comparable to opt pipeline
- Plugin passes only **nameable in** `-passes=`
- Target pipeline needs entry-point callbacks or something comparable
- `CodeGenPassBuilder::insertPass/disablePass` exist, but no replace/reorder



Next episode: Philipp Grulich will talk about **Nautilus**, the lightweight, tracing JIT on **December 10**

github.com/nebulastream/nautilus